

Objectif

Comprendre comment fonctionnent les différentes méthodes d'authentification d'un service REST. Notamment via les APIKey, l'authentification BASIC et l'authentification par jeton au porteur.

1 Utiliser

L'API <https://detectlanguage.com> est une API de détection de la langue utilisée dans un texte. On lui envoie une phrase en paramètre et elle renvoie le langage supposé avec un pourcentage de crédibilité.

Cette API est implémentée avec 2 méthodes d'authentification : `Basic` (on passe le login et le mdp) ou `Bearer` (on passe un jeton "au porteur")

1. Effectuer (soit avec curl, soit avec httpie) une requête GET sur l'un des endpoint publics de cette API

```
http https://ws.detectlanguage.com/0.2/languages
curl https://ws.detectlanguage.com/0.2/languages
```

Aucune authentification n'est demandée, la réponse arrive classiquement

2. Effectuer maintenant une requête GET mais sur un endpoint privé

```
http https://ws.detectlanguage.com/0.2/detect q='Hello World'
curl https://ws.detectlanguage.com/0.2/detect -d q='Hello world'
```

Comme nous ne sommes pas identifiés, l'API renvoie une erreur 401 Unauthorized

3. Pour être authentifié, il est nécessaire de créer un compte avec login/mdp afin d'être connu par l'application. Connectez vous sur <https://detectlanguage.com>. Pour éviter de créer une nouvelle identité, prendre le SignIn with Google ou Facebook.

Vous récupérez alors un token fourni par Google. Le site detectlang en a profité pour créer un compte associé à ce token. Quelque chose comme `f3aee63dd55b575d23c928cdd55fe2fe`.

Ne perdez pas ce token !

Ce type de jeton est dit "au porteur" car il ne vous identifie pas. Vous pourriez par exemple le passer à votre voisin. Il a par contre une durée de vie limitée !

4. On reprend la même requête mais avec le token cette fois

```
http https://ws.detectlanguage.com/0.2/detect Authorization:"Bearer <token>" q='Hello W
curl https://ws.detectlanguage.com/0.2/detect -H "Authorization: Bearer <token>" -d q="H
```

5. Ré-essayez en enlevant 1 ou 2 caractères au token, pour vérifier que cette fois l'accès ne fonctionne plus.
6. D'autres endpoints de ce site nécessitent aussi l'apport du token. Testez-en un :

```
http https://ws.detectlanguage.com/0.2/user/status Authorization:"Bearer <token>"
curl https://ws.detectlanguage.com/0.2/user/status -H "Authorization: Bearer <token>"
```

7. Comme vous êtes maintenant connus du site, et que celui-ci autorise aussi l'identification BASIC via sa page de login, nous pouvons maintenant tester cette autre technique :

(a) accédez avec votre navigateur à l'url

```
https://ws.detectlanguage.com/0.2/detect?q=The+world+is+really+beautiful
```

(b) une popup gérée par le navigateur s'affiche

(c) passez le token comme identifiant, sans mot de passe (c'est leur choix)

(d) Faites plusieurs requêtes, le token n'est plus demandé et est maintenant géré par le navigateur

2 Réaliser sa propre authentification

Cette partie a pour objectif de montrer comment sécuriser une application REST par jeton. Nous aborderons 3 techniques différentes : par Apikey, par authentification BASIC, et pour finir par un jeton au porteur de type JWT.

Pour cela nous avons besoin d'un contrôleur REST trivial (peu importe ce qui est implémenté, il faut juste un endpoint à protéger) ...

```
@WebServlet("/etudiants/*")
public class ControleurEtudiant extends HttpServlet {

    public void doGet(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {
        res.setContentType("application/json;charset=UTF-8");
        PrintWriter out = res.getWriter();
        out.println("retour d'un GET");
        out.close();
    }
}
```

et d'une table de base de données. Là aussi on va au plus simple.

```
CREATE TABLE users(login text, pwd text, ROLE text, token text, nb int);
INSERT INTO users VALUES ('jean', 'jean', NULL, NULL, 0);
INSERT INTO users VALUES ('paul', 'paul', NULL, NULL, 0);
```

2.1 Aucune authentification

Le contrôleur REST mis en place ne contient aucune authentification particulière. On peut donc accéder à son endpoint directement.

1. Testez l'accès à votre api

```
curl -X GET http://localhost:8080/tp333/etudiants
```

2.2 Avec APIKey (version 1)

La plus simple des techniques d'identification REST consiste à mettre en place un token permanent propre à l'API : une API key. Ce type de token doit être généré une fois pour l'utilisateur, qui l'utilisera ensuite dans toutes ses requêtes. C'est exactement ce qui se produisait avec <https://detectlanguage.com>. Pour cela il faut s'y prendre en 5 étapes

1. Réalisez une page `login1.html` qui contient un formulaire d'identification avec `login`, `pwd`. Ce formulaire appellera une servlet `GenererToken1.java`
2. Réalisez la servlet `GenererToken1.java` en 5 étapes
 - (a) Récupérez les paramètres `login` et `pwd`
 - (b) Vérifiez que l'utilisateur existe dans la base
 - (c) Fabriquez le token. Le plus simple des tokens est obtenu en générant un **UUID** (fourni dans `java.util`).
 - (d) Enregistrez le token dans la table `users` pour cet utilisateur
 - (e) Affichez le token à l'utilisateur s'il est connu ou un message `Vous êtes inconnu ici !` dans le cas contraire.
3. Recopiez maintenant la servlet `ControleurEtudiant.java` en `ControleurEtudiant1.java`. Modifiez cette servlet pour qu'elle possède une méthode booléenne `verifToken` qui sera appelée en début de requête `GET`.
4. Ecrire la méthode `verifToken` pour qu'elle effectue 3 étapes
 - (a) Lire le token passé en paramètre de la requête `HTTP`.
 - (b) Vérifier dans la base que le token a bien été enregistré pour quelqu'un (ce qui permettrait par la même occasion de savoir à qui il appartient)
 - (c) Renvoyer un booléen indiquant si tout est correct ou pas.
5. Modifiez le `doGet` pour qu'il appelle `verifToken` et renvoie le résultat de la requête `GET` ou une erreur 401 si nécessaire.

L'API est fin prête à être testée

1. <http://localhost:8080/tp333/login1.html> pour afficher le token
2. récupérez le token,
3. puis réalisez n requêtes de la forme

```
http http://localhost:8080/tp333/etudiants1 token==<letoken>
curl "http://localhost:8080/tp333/etudiants1?token=<letoken>"
```

2.3 Avec authentification BASIC (version 2)

L'authentification BASIC est aussi très simple : elle consiste à fabriquer un token sous la forme `base64(login:pwd)`. Ce type de token est en général passé par le header `Authentication:` suivi du mot clé `Basic` puis du token.

`Authorization:Basic <letoken>`

Base64 étant réversible, ce token peut être décodé par le contrôleur qui récupère le login et le pwd de l'utilisateur et vérifie s'il est valide.

1. Faites une copie de `login1.html`, `ControleurEtudiant1.java` et `GenererToken1.java` en les renommant tous avec un 2.
2. Modifiez `login2.html` pour qu'il appelle `GenererToken2.java`
3. Modifiez la servlet `GenererToken2.java` pour qu'elle réalise 4 étapes
 - (a) Récupérer les paramètres `login` et `pwd`
 - (b) Vérifier que l'utilisateur existe dans la base
 - (c) Fabriquer le token à l'aide de la classe `java.util.Base64` qui fournit un encodeur.

Cette fois on ne le range plus dans la base, c'est inutile.

- (d) Affiche le token à l'utilisateur s'il est connu ou un message d'erreur dans le cas contraire.
4. Modifiez la méthode booléenne `verifToken` de la servlet `ControleurEtudiant2.java` pour qu'elle effectue 4 étapes
 - (a) Lire le token dans l'entête HTTP
 - (b) Décoder le token afin de récupérer `login` et `pwd`
 - (c) Vérifier que l'utilisateur concerné existe bien
 - (d) Renvoyer un booléen indiquant si tout est correct ou pas.
5. Modifiez le `doGet` pour qu'il appelle `verifToken` et renvoie le résultat de la requête GET ou une erreur 401 si nécessaire.

L'API est fin prête à être testée

1. <http://localhost:8080/tp333/login2.html> pour afficher le token
2. Récupérer le token,
3. puis *n* requêtes de la forme

```
http http://localhost:8080/tp333/etudiants2 Authorization:"Basic <letoken>"
curl "http://localhost:8080/tp333/etudiants2" -H "Authorization: Basic <letoken>"
```

ou plus simple

```
http -a <login:pass> http://localhost:8080/tp333/etudiants2
curl -u <login:pass> http://localhost:8080/tp333/etudiants2
```

4. Appelez la page <http://localhost:8080/tp333/etudiants2> avec votre navigateur. Dans le cas d'une authentification `Basic`, dès que le navigateur reçoit une erreur 401 il lance automatiquement une popup d'authentification.

2.4 Compréhension du token JWT

JWT est un token signé standardisé constitué de 3 parties encodées en Base64 et séparées par un point. La 3^e partie contient une signature cryptographique afin qu'il soit infalsifiable. Il a la forme suivante :

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJwaGlzaXBwZS5tYXRoaWV1QHVuaXYtbGlzYUuZnIiLCJzZW50IGxvcnMgZHUgVFAgMzMzIiwibmFtZSI6IiBoaWxpcHB1IE1hdGhpZXUiLCJpYXQiOiE3MDkyNDc2MDAsImV4cCI6MTc0MzQ1ODQwMH0.8sjfsa6H_B2PitvAH-sfavFA5d2EZmPSR-W4CZknzSc
```

1. Utilisez le site <https://www.base64decode.org/> sur chacune des parties pour les décoder.

On constate que 2 des 3 parties sont publiques. Le payload (partie du milieu) peut contenir des informations personnalisées. Le `sub` contient en général un identifiant de personne, `iat` la date d'émission, `exp` la date d'expiration du token (fournie en epoch. convertisseur : <https://www.epochconverter.com/>) etc ...

2. Jusqu'à quand le token ci dessus est-il valide ? quel est l'identifiant de l'émetteur ?
3. La signature a été fabriquée avec une clé secrète concaténée aux deux autres parties. On ne peut pas la décoder. Mais par contre, si on connaît cette clé secrète, on peut régénérer l'ensemble : on refait un HMACSHA256 sur chacune des parties et on retrouve le token.

Sans cette clé vous ne pouvez rien faire. Demandez la à votre enseignant. En mettant le header et le payload que vous avez déchiffré précédemment puis la bonne signature, vous êtes en mesure de régénérer le token, et donc voir s'il est correct.

Refabriquez le token à l'aide de <https://jwt.io/> et vérifiez qu'il est bien identique à celui ci-dessus.

C'est cette procédure qui doit être réalisée à chaque requête.

2.5 Avec token JWT (version 3)

Les tokens complexes comme JWT sont en général passés dans le header avec le mot clé `Bearer`

```
Authorization:Bearer <letoken>
```

Il existe plusieurs librairies qui facilitent la gestion des tokens JWT. nous prendrons ici la librairie `jjwt` qui est fournie sous la forme de 3 jars distincts : `jjwt-api-0.12.5.jar` , `jjwt-impl-0.12.5.jar` , `jjwt-jackson-0.12.5.jar`

Téléchargez ces jars, faites en sorte qu'ils soient accessibles par tomcat (`tomcat/lib`) et par votre outil de compilation.

Malgré l'aide fournie par `jjwt`, la création et le décodage d'un token reste une tâche complexe. Récupérez l'objet `JwtManager.java` qui contient 2 méthodes statiques `createJWT()` et `decodeJWT(token)`.

1. Faites une copie de `login2.html`, `ControleurEtudiant2.java` et `GenererToken2.java` en les renommant tous avec un 3.
2. Modifiez `login3.html` pour qu'il appelle `GenererToken3.java`
3. Modifiez la servlet `GenererToken3.java` de la manière suivante :
Remplacez la génération du token en Base64 par `JwtManager.createJWT()` ; (C'est juste le token qui change ...)

On ne range toujours pas le token dans la base, c'est inutile.

4. Modifiez la méthode `verifToken` de la servlet `ControleurEtudiant3.java` de la manière suivante :
Remplacez la génération du token en Base64 par `Claims claims = JwtManager.decodeJWT(token)` ;
Cette méthode génère une exception si le token est invalide (périmé par ex).

L'API est fin prête à être testée

1. <http://localhost:8080/tp333/login3.html> pour afficher le token
2. Récupérer le token,
3. Testez le token sur <https://jwt.io/>
4. puis réalisez n requêtes de la forme

```
http http://localhost:8080/tp333/etudiants3 Authorization:"Bearer <letoken>"
```

```
curl "http://localhost:8080/tp333/etudiants3" -H "Authorization: Bearer <letoken>"
```

5. Le token créé a une durée de validité de 4mn attendez 4mn et refaites une requête. Cette fois l'accès n'est plus autorisé